

Quantum Computing Algorithms and Applications

Homework 2

Due by May 14, 2026 (Thursday)

Note: You have to include the code you used to solve the problems in your assignment.

Important: Please use your **student ID number** as the random seed in all problems. For example: `seed = 0` (replace 0 with the numerical part of your student ID). This ensures that each student's results are unique and reproducible. **Failure to set the random seed will result in a penalty of 50% for each of Problems 1 to 3.**

Problem 1: 01 Knapsack — QUBO and Quantum Annealing

One of the renowned combinatorial optimization problems is the 01 knapsack problem, which is NP-complete. Consider 10 items to be packed into a knapsack with a maximum weight capacity of $W = 165$. The goal is to select items that maximize the total value without exceeding the weight limit. The items with their respective weights and values are given in [Table 1](#).

Table 1: Items with their respective weights and values

Item	1	2	3	4	5	6	7	8	9	10
Weight	23	31	29	44	53	38	63	85	89	82
Value	92	57	49	68	60	43	67	84	87	72

The 01 knapsack problem can be formulated as:

$$\max_{\mathbf{x}} \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}, \quad (1)$$

where v_i and w_i are the value and weight of item i , $W = 165$ is the maximum capacity, and x_i indicates whether item i is selected.

To solve this on a quantum computer, we first reformulate it as a Quadratic Unconstrained Binary Optimization (QUBO) problem by moving the constraint into the objective with a penalty:

$$\min_{\mathbf{x}} \left[-\sum_{i=1}^n v_i x_i + \lambda \left(\sum_{i=1}^n w_i x_i - W \right)^2 \right], \quad (2)$$

where $\lambda > 0$ is a penalty coefficient that enforces the weight constraint. Note that the inequality constraint must be converted to an equality constraint, e.g., by introducing slack variables (see [Appendix A](#) for details on two common approaches). Hint and example codes can be found in Ref. [1].

Here is sample code for solving a QUBO with D-Wave's Ocean SDK (see [Appendix E](#) for installation):

```
import dimod
import neal
import numpy as np

seed = 0 # replace with your student ID
np.random.seed(seed)

# Construct QUBO matrix (dict format)
Q = {} # {(i, j): Q_ij}
# Q[(0, 0)] = ... # diagonal terms
# Q[(0, 1)] = ... # off-diagonal terms
# ... fill in from your QUBO derivation
```

```

bqm = dimod.BQM.from_qubo(Q)

# Exact solver (classical, for verification)
exact_sampler = dimod.ExactSolver()
exact_result = exact_sampler.sample(bqm)

# Simulated annealing
sa_sampler = neal.SimulatedAnnealingSampler()
sa_result = sa_sampler.sample(bqm, num_reads=1000, seed=seed)

```

Please include the following in your report:

- Solve the knapsack problem using a classical method (brute force or dynamic programming). Report the optimal combination of items, the total weight, and the total value. Then derive the QUBO formulation: clearly show how you handle the inequality constraint and write out the resulting Q matrix entries.
- Solve the QUBO using D-Wave's `ExactSolver` and verify it matches the classical optimal. Investigate the effect of the penalty coefficient λ : try at least 3 values and for each report whether the solution is feasible and optimal. Discuss how to choose an appropriate λ (see [Appendix E](#)).
- Solve with `SimulatedAnnealingSampler`. Investigate `num_reads` with at least 4 values (e.g., 10, 100, 1000, 10000). Report the success probability (fraction of reads returning the optimal bitstring found in (b)) for each.
- Provide a comparison table: method (classical, exact QUBO, simulated annealing), best value found, total weight, feasibility, and computation time. Write a brief discussion (3–5 sentences).

Problem 2: Max-Cut with QAOA

The Maximum Cut (Max-Cut) problem is another classic combinatorial optimization problem: given an undirected graph $G = (V, E)$, partition the vertices into two sets such that the number of edges crossing the partition is maximized. Unlike the knapsack problem, Max-Cut is *unconstrained* — every partition is feasible — making it a natural testbed for the Quantum Approximate Optimization Algorithm (QAOA) [2].

The Max-Cut cost function in Ising variables $z_i \in \{-1, +1\}$ is:

$$C(\mathbf{z}) = \frac{1}{2} \sum_{(i,j) \in E} (1 - z_i z_j), \quad (3)$$

which counts the number of edges between opposite partitions. The corresponding cost Hamiltonian is:

$$H_C = -\frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j + \frac{|E|}{2} \mathbb{I}, \quad (4)$$

where minimizing H_C is equivalent to maximizing the cut. Note that Max-Cut maps directly to an Ising Hamiltonian without the QUBO-to-Ising conversion needed for the knapsack — this is because the binary partition $\{-1, +1\}$ is already in spin form.

The QAOA circuit alternates cost and mixer unitaries [3]:

$$|\gamma, \beta\rangle = U(H_M, \beta_p) U(H_C, \gamma_p) \cdots U(H_M, \beta_1) U(H_C, \gamma_1) |+\rangle^{\otimes n}, \quad (5)$$

where $U(H_C, \gamma) = e^{-i\gamma H_C}$ implements the cost evolution, $U(H_M, \beta) = e^{-i\beta H_M}$ implements the mixer (standard choice: $H_M = \sum_i X_i$), and the variational parameters $\{\gamma_k, \beta_k\}_{k=1}^p$ are optimized classically.

Use the following code to generate your **student-specific** random graph:

```

import networkx as nx
import numpy as np

seed = 0 # replace with your student ID
G = nx.gnp_random_graph(n=8, p=0.5, seed=seed)
print(f"Nodes: {G.number_of_nodes()}, Edges: {G.number_of_edges()}")

```

For the QAOA implementation, see the PennyLane QAOA tutorial [3] which uses Max-Cut as its example.

Please include the following in your report:

- (a) Generate your random graph with your student ID seed. Visualize it and find the exact Max-Cut value by brute-force enumeration (only $2^8 = 256$ partitions). Report the graph's edges, the maximum cut value, and the optimal partition(s).
- (b) For QAOA depth $p = 1$, plot the 2D energy landscape $F(\gamma, \beta) = \langle \gamma, \beta | H_C | \gamma, \beta \rangle$ as a heatmap over $\gamma \in [0, 2\pi]$ and $\beta \in [0, \pi]$. Identify the global minimum in the landscape. Does the optimizer reliably find it?
- (c) Implement QAOA and solve your Max-Cut instance for depths $p \in \{1, 2, 3, 4\}$. For each depth, report: the best cut found, the approximation ratio (found cut / optimal cut), and the optimized parameters.
- (d) Also solve with `SimulatedAnnealingSampler` for comparison. Provide a comparison table across all methods (brute-force, simulated annealing, QAOA at each depth): best cut, approximation ratio, and computation time. Write a brief discussion (3–5 sentences).

Problem 3: Low Autocorrelation Binary Sequences (LABS)

The Low Autocorrelation Binary Sequences (LABS) problem is a notoriously difficult combinatorial optimization problem with critical applications in high-performance radar, telecommunications, and cryptography. Given a binary sequence $\mathbf{s} = (s_1, \dots, s_N)$ with $s_i \in \{-1, +1\}$, define the aperiodic autocorrelations and sidelobe energy:

$$C_k(\mathbf{s}) = \sum_{i=1}^{N-k} s_i s_{i+k}, \quad E(\mathbf{s}) = \sum_{k=1}^{N-1} C_k(\mathbf{s})^2, \quad (6)$$

and the *merit factor*:

$$F(\mathbf{s}) = \frac{N^2}{2E(\mathbf{s})}. \quad (7)$$

The goal is to find $\mathbf{s}$ that **maximizes** F (equivalently, minimizes E).

LABS is harder than Max-Cut in two ways: (i) the cost contains nonlocal 2-spin and 4-spin interactions (E includes $s_i s_{i+k} s_j s_{j+k}$ terms after expanding the squared autocorrelations), so standard 2-local QAOA does not directly apply; and (ii) the energy landscape is glassy with exponentially many local minima [4]. The best known classical approach is Memetic Tabu Search (MTS) [5], and recent work has shown that *quantum-enhanced* MTS — where quantum-generated samples seed the classical tabu search — achieves state-of-the-art scaling for LABS [6].

In this problem, you will **design, implement, and benchmark** at least **two advanced quantum or hybrid strategies** for LABS. You are not restricted to QAOA; any quantum algorithm (variational or otherwise), classical-quantum hybrid pipeline, or novel architecture of your own design is allowed, provided the quantum component plays a non-trivial role. See [Appendices C and D](#) for implementation guidance, including quartic-Hamiltonian tips and advanced QAOA techniques.

Suggested strategies (implement at least 2; you may also propose your own):

- **Quartic-Hamiltonian QAOA:** Build H_C as a sum of Pauli- Z products of degree up to 4 and run standard QAOA. Compare different depths p .
- **Quadratization:** Introduce auxiliary variables for spin products and enforce them with standard 2-local QUBO gadgets, reducing the quartic objective terms to 2-local at the price of more qubits. Solve with QAOA, VQE, or quantum annealing.
- **Variational Quantum Eigensolver (VQE):** Use a hardware-efficient ansatz (e.g., layered R_Y rotations with CNOT entanglers) instead of the QAOA ansatz, and optimize the expectation value of H_C^{LABS} .
- **Quantum annealing (simulated):** Quadratize LABS and solve with D-Wave's samplers as in [Problem 1](#).

- **Warm-starting / INTERP / RQAOA:** Apply the advanced QAOA techniques from [Appendix C](#) to the LABS problem.
- **Classical optimizer comparison:** Compare at least 3 classical optimizers (gradient descent, COBYLA, Nelder-Mead, SPSA, Adam) for the variational outer loop.
- **Hybrid quantum-classical search:** Use quantum samples (from QAOA, VQE, or annealing) to seed classical local search (e.g., Tabu or simulated annealing), then feed the improved solution back into the next quantum iteration. See the quantum-enhanced MTS approach of Ref. [6].
- **Novel approach:** Any other quantum or hybrid strategy you design. Creativity is encouraged.

Benchmark Target

Solve LABS for sequence length $N = 20$. The known optimum is $E^* = 26$ (merit factor $F^* = 400/52 \approx 7.692$); verify this against published tables or by reproducing it with your solver. Define the merit factor ratio $r = F_{\text{best}}/F^*$ and the budget N_{eval} as the total number of quantum circuit evaluations spent during your entire hybrid pipeline.

Your best strategy should achieve $r \geq 0.85$ using $N_{\text{eval}} \leq 5000$. For $N = 20$, this threshold effectively requires finding an optimal sequence with $E = 26$, since the next energy level $E = 34$ has only $r \approx 0.765$. For orientation, a single random sequence has expected $r \approx 0.16$, while the best sequence among 5000 random samples is typically around $r \approx 0.6$; standard QAOA or naive VQE at low depth should be treated as baselines and compared empirically on this instance. Try to get higher merit factor ratios, smaller budgets, thoughtful analysis of the r -vs- N_{eval} trade-off, and implement novel hybrid strategies (e.g., quantum-seeded classical local search, generative quantum eigensolver (GQE)).

Please include the following in your report:

- Implement the LABS cost function $E(\mathbf{s}) = \sum_{k=1}^{N-1} C_k(\mathbf{s})^2$ and merit factor $F(\mathbf{s}) = N^2/(2E(\mathbf{s}))$. Verify your implementation on the Barker sequence for $N = 11$ (sequence $+++---+-+--$, whose autocorrelations give $E = 5$ and $F = 12.10$ under this convention). State which two (or more) strategies you chose and how you encode LABS into a Hamiltonian (quartic or quadratized).
- Implement each strategy and apply it to LABS with $N = 20$. Report: the best sequence found, E_{best} , F_{best} , the merit factor ratio r , and N_{eval} for each.
- Provide a comparison table of your strategies alongside two baselines: (i) random sampling with the same total shot budget, and (ii) a purely classical baseline (e.g., simulated annealing from [Problem 1](#)'s workflow or simple Tabu search). Also plot convergence curves (cost vs. iteration) for each strategy on the same axes.
- Write a discussion (5–8 sentences): which strategy works best on LABS and why? How does the glassy, quartic landscape of LABS differ from the quadratic Max-Cut landscape of [Problem 2](#)? What design choices have the largest impact? Relate your findings to Ref. [4] or Ref. [6].

References

- [1] Alejandro Montanez. Quadratic unconstrained binary optimization (QUBO). https://pennylane.ai/qml/demos/tutorial_QUBO, February 2024.
- [2] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.
- [3] Jack Ceroni. Intro to QAOA. https://pennylane.ai/qml/demos/tutorial_qaoa_intro, 2020.
- [4] Ruslan Shaydulin, Changhao Li, Shouvanik Chakrabarti, Matthew DeCross, Dylan Herman, et al. Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem. *Science Advances*, 10(22):eadm6761, 2024.

- [5] Zhiwei Zhang, Jiayu Shen, Niraj Kumar, and Marco Pistoia. New improvements in solving large LABS instances using massively parallelizable memetic tabu search. *arXiv preprint arXiv:2504.00987*, 2025.
- [6] Alejandro Gomez Cadavid, Pranav Chandarana, Sebastián V. Romero, Jan Trautmann, Enrique Solano, et al. Scaling advantage with quantum-enhanced memetic tabu search for LABS. *arXiv preprint arXiv:2511.04553*, 2025.
- [7] Stuart Hadfield, Zhihui Wang, Bryan O’Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

A QUBO Formulation Details

To handle the knapsack inequality constraint $\sum_i w_i x_i \leq W$, two common approaches exist:

Slack Variable Method

Introduce binary slack variables $s_0, s_1, \dots, s_{M-1}$ to convert the inequality to an equality:

$$\sum_{i=1}^n w_i x_i + \sum_{k=0}^{M-1} 2^k s_k = W, \quad (8)$$

where $M = \lceil \log_2(W) \rceil$ is the number of slack bits needed. The QUBO objective becomes:

$$\min_{\mathbf{x}, \mathbf{s}} \left[- \sum_{i=1}^n v_i x_i + \lambda \left(\sum_{i=1}^n w_i x_i + \sum_{k=0}^{M-1} 2^k s_k - W \right)^2 \right]. \quad (9)$$

This method is exact but increases the number of binary variables (and hence qubits) by M . For $W = 165$, we need $M = \lceil \log_2(165) \rceil = 8$ slack variables, giving $10 + 8 = 18$ total binary variables.

Unbalanced Penalization

An alternative approach avoids slack variables by using an asymmetric penalty [1]:

$$P(\mathbf{x}) = \lambda_1 \left(\sum_{i=1}^n w_i x_i - W \right) + \lambda_2 \left(\sum_{i=1}^n w_i x_i - W \right)^2, \quad (10)$$

where $\lambda_1 > 0$ adds a linear penalty that discourages overweight solutions even without slack variables. This reduces the number of qubits to just 10 (items only) but requires careful tuning of λ_1 and λ_2 .

B QUBO to Ising Conversion

The QUBO formulation uses binary variables $x_i \in \{0, 1\}$, while quantum hardware naturally works with spin variables $z_i \in \{-1, +1\}$. The conversion uses the substitution $x_i = (1 - z_i)/2$. Substituting into the QUBO objective $\sum_{i \leq j} Q_{ij} x_i x_j$ yields the Ising Hamiltonian:

$$H_C = \sum_{i < j} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{const}, \quad (11)$$

where:

$$J_{ij} = \frac{Q_{ij}}{4}, \quad (12)$$

$$h_i = -\frac{Q_{ii}}{2} - \sum_{j \neq i} \frac{Q_{ij}}{4}. \quad (13)$$

In code, you can use `dimod`’s built-in conversion:

```
import dimod
bqm = dimod.BQM.from_qubo(Q)
linear, quadratic, offset = bqm.to_ising()
# linear: {i: h_i}, quadratic: {(i, j): J_ij}
```

Note: Max-Cut (Problem 2) maps directly to an Ising Hamiltonian without this conversion — the cost Hamiltonian is simply $H_C = -\frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j$ plus a constant.

C Advanced QAOA Strategies

This appendix provides brief descriptions of several strategies for Problem 3. For a comprehensive treatment, see Ref. [7].

Warm-Starting QAOA

Instead of initializing all qubits in $|+\rangle$, prepare an initial state biased toward a known good classical solution $\mathbf{x}^*$. For each qubit i , apply $R_Y(\theta_i)$ with $\theta_i = 2 \arcsin(\sqrt{c_i^*})$, where $c_i^* = \epsilon$ if $x_i^* = 0$ and $c_i^* = 1 - \epsilon$ if $x_i^* = 1$, with $\epsilon \in (0, 0.5)$ controlling the bias strength. The mixer must also be adapted so that the warm-started state is its ground state. The standard choice (Egger et al.) is

$$H_M = \sum_i [-\sin(\theta_i) X_i - \cos(\theta_i) Z_i]. \quad (14)$$

Sanity check: at $\theta_i = 0$ (i.e. $c_i^* = 0$, initial state $|0\rangle$), $H_M^{(i)} = -Z_i$, whose ground state is $|0\rangle$, as required.

Constraint-Preserving Mixers

For constrained problems like the knapsack, the standard X -mixer can generate infeasible states. A constraint-preserving mixer restricts transitions to the feasible subspace. One approach for binary constraints: use pairwise XY -type mixers $H_{ij}^{XY} = \frac{1}{2}(X_i X_j + Y_i Y_j)$ that swap excitations between qubits without changing the total Hamming weight. See Ref. [7] for a detailed construction.

Parameter Transfer (INTERP)

Initialize depth- $(p+1)$ QAOA using the optimized parameters from depth- p via the Zhou et al. INTERP scheme:

$$\gamma_i^{[p+1]} = \frac{i-1}{p} \gamma_{i-1}^{[p]} + \frac{p-i+1}{p} \gamma_i^{[p]}, \quad i = 1, \dots, p+1, \quad (15)$$

with boundary conditions $\gamma_0^{[p]} = \gamma_{p+1}^{[p]} = 0$ (and analogously for β_i). For example, going from $p = 1$ to $p = 2$ gives $\gamma_1^{[2]} = \gamma_2^{[2]} = \gamma_1^{[1]}$. This linear interpolation leverages the insight that good parameters form smooth schedules across depths. Compare convergence speed and final solution quality with and without transfer.

Recursive QAOA (RQAOA)

After running standard QAOA at a given depth, compute the expectation values $\langle Z_i Z_j \rangle$ for all variable pairs. Fix the pair with the strongest correlation (i.e., largest $|\langle Z_i Z_j \rangle|$): if positive, set $z_i = z_j$; if negative, set $z_i = -z_j$. Substitute this constraint into the Hamiltonian, reducing the problem by one variable, and recurse. This iteratively simplifies the problem until all variables are determined.

D LABS Implementation Notes

Quartic Cost Hamiltonian

The LABS sidelobe energy $E(\mathbf{s}) = \sum_k C_k(\mathbf{s})^2$ expands into a sum of terms with degree at most 4:

$$E(\mathbf{s}) = \sum_{k=1}^{N-1} \sum_{i,j=1}^{N-k} s_i s_{i+k} s_j s_{j+k}. \quad (16)$$

Under the substitution $s_i \rightarrow Z_i$, the cost Hamiltonian becomes:

$$H_C^{\text{LABS}} = \sum_{k=1}^{N-1} \sum_{i,j=1}^{N-k} Z_i Z_{i+k} Z_j Z_{j+k}. \quad (17)$$

PennyLane can construct this as a sum of **Tensor** products of **PauliZ** operators. The cost-layer evolution $e^{-i\gamma Z_i Z_{i+k} Z_j Z_{j+k}}$ decomposes into a network of CNOTs and an $R_Z(2\gamma)$ on an ancilla-free path; see `qml.exp` or `qml.evolve`. Before implementation, simplify repeated Pauli factors: terms with $i = j$ reduce to identity ($Z_i^2 = I$), while overlapping pairs such as $i + k = j$ reduce to 2-local terms. Identity terms add the constant $\sum_{k=1}^{N-1} (N - k) = N(N - 1)/2$ to E ; this constant may be dropped during optimization, but it must be restored when reporting E and F .

Quadratization Alternative

Introduce auxiliary variables $y_{ij} = s_i s_j$ so that each C_k^2 becomes quadratic in the product variables. The constraints $y_{ij} = s_i s_j$ must also be quadratized: the direct spin penalty $\lambda(1 - s_i s_j y_{ij})^2/4$ is 3-local, not a QUBO term. A valid 2-local route is to convert spins to binary variables and use standard product/XOR gadgets (or a library quadratizer such as `dimod.make_quadratic`) to enforce the auxiliary relations. This reduces the problem to a standard QUBO/Ising at the price of $O(N^2)$ additional variables. For $N = 20$, this requires ~ 210 qubits — not simulable classically, but the structure is suitable for quantum annealing on D-Wave hardware.

Verifying Your Implementation

Before running on $N = 20$, verify your LABS solver on small instances:

- $N = 5$: optimal $E^* = 2$, $F^* = 6.25$ (e.g., $\mathbf{s} = +++-+$).
- $N = 11$ (Barker): optimal $E^* = 5$, $F^* = 12.10$ (unique up to symmetry).
- $N = 13$ (Barker): optimal $E^* = 6$, $F^* \approx 14.083$.

E Practical Tips

Choosing the Penalty Coefficient λ

The penalty coefficient λ must be large enough to make constraint violations costly, but not so large that it dominates the objective and flattens the energy landscape. A common heuristic is $\lambda > \max_i v_i / \min_i w_i$. For the given dataset, this gives $\lambda > 92/23 = 4$. Start with this estimate and adjust based on whether the solver finds feasible solutions.

QAOA Energy Landscape ([Problem 2](#))

For $p = 1$, the QAOA has exactly two parameters (γ, β) , allowing a full 2D visualization:

```
import numpy as np
gammas = np.linspace(0, 2 * np.pi, 50)
betas = np.linspace(0, np.pi, 50)
landscape = np.zeros((len(gammas), len(betas)))
for i, g in enumerate(gammas):
    for j, b in enumerate(betas):
        landscape[i, j] = cost_circuit(np.array([[g], [b]]))
# Plot with plt.imshow or plt.contourf
```

Look for global vs. local minima, symmetries, and flat regions. Relate the landscape structure to how easily the classical optimizer converges.

Installing D-Wave Ocean SDK

```
pip install dwave-ocean-sdk
```

This installs `dimod`, `neal`, `dwave-system`, and other tools. For optional D-Wave hardware access, run `dwave setup` and enter your API token from <https://cloud.dwavesys.com>.